

GearShift LLM: Dynamic Model Switching for Efficient Generative AI Inference

Modeling and Scaling of Generative AI Systems

Group 9: Ranim Ourari

Autumn semester 2025

Code Repository

All code used in this project is available at: <https://github.com/RanimOurari/GearShift-LLM>

Abstract

Large Language Models (LLMs) generate high-quality text but suffer from high inference latency under sustained load, where queueing delays quickly dominate response time. This project studies GearShift LLM, a dynamic inference strategy that improves scalability by switching between a large, high-quality model (GPT-2) and a smaller, faster model (DistilGPT-2) based on system load. We first conduct a static analysis by measuring inference latency, service-time variability, and generation quality, and use these measurements to build an analytic M/G/1 queueing model for latency prediction. We then simulate an online inference service with Poisson arrivals and FIFO scheduling to evaluate deterministic and probabilistic switching policies under different loads. The results show that a simple queue-length-based policy significantly reduces latency while preserving high output quality, and that the analytic M/G/1 model closely matches simulated performance.

1 Introduction

Large Language Models (LLMs) deliver high-quality text generation but pose significant challenges for real-time serving due to high inference latency under increasing load. As request rates grow, queueing effects dominate system behavior, causing latency to rise rapidly. This project investigates *GearShift LLM*, a dynamic serving strategy that switches between a large, high-quality model (GPT-2) and a smaller, faster model (DistilGPT-2) based on queue length. Using empirical latency measurements, an online queue simulator, and quality-aware policy optimization, we evaluate how dynamic switching impacts latency and output quality under varying arrival rates. We further validate system behavior using an analytic M/G/1 queueing model, demonstrating that simple queue-based control policies can achieve modest but consistent latency reductions while preserving high output quality.

2 Generative AI Application

GearShift LLM uses two HuggingFace models:

- **Large model:** GPT-2: higher quality, slower.

- **Small model:** DistilGPT-2: faster but with reduced generation quality.

Both models are executed on a Kaggle P100 GPU using PyTorch. To quantify the quality trade-off between the large and small models, we evaluate semantic similarity between generated responses rather than relying on task-specific accuracy metrics. Since the prompts are open-ended and lack ground-truth labels, we adopt a task-agnostic semantic evaluation approach.

For a fixed set of ten everyday questions, responses are generated using both GPT-2 and DistilGPT-2. Each response is encoded using the all-MiniLM-L6-v2 SentenceTransformer, and cosine similarity between embeddings is computed. This metric serves as a proxy for semantic consistency, capturing whether the smaller model preserves the meaning of the large model’s output.

GPT-2 is treated as a reference baseline with a normalized quality score of 1.0. DistilGPT-2 achieves an average similarity score of 0.77, indicating that while its responses differ in phrasing and detail, they remain semantically aligned with those of the large model. These per-model quality scores are treated as intrinsic model properties and are later combined with model usage statistics to compute system-level quality during policy optimization.

3 Statistical Analysis

This section describes the dataset, models, measurement procedure, and analysis of the offline latency benchmarks.

3.1 Dataset and Experimental Setup

We extracted 200 question titles from the Yahoo Answers Topics dataset. These were tokenized and grouped into approximate lengths of 16, 32, and 64 tokens. For each prompt length and each batch size in $\{1, 4, 8\}$, we sampled prompts, warmed up the model, and measured latency for generating 20 new tokens.

Each experiment was repeated five times. A sample of the resulting latency dataset is shown in Table 1.

Table 1: Example rows from `latency_measurements.csv`.

Model	Prompt Length	Batch Size	Mean Latency (s)	Std Dev (s)
GPT-2	16	1	0.1738	0.0024
GPT-2	16	4	0.0608	0.0094
GPT-2	16	8	0.0257	0.0002
DistilGPT-2	16	1	0.1016	0.0026
DistilGPT-2	16	8	0.0148	0.0004

3.2 Latency vs Batch Size

Figure 1 shows the average latency for both models across batch sizes.

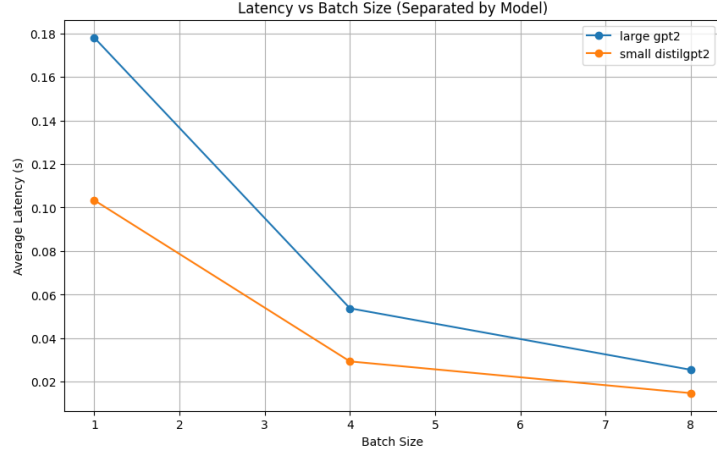


Figure 1: Latency vs Batch Size for GPT-2 and DistilGPT-2.

Latency decreases sharply as batch size increases. GPT-2 drops from 0.18 s at batch size 1 to 0.029 s at batch size 8 and DistilGPT-2 drops from 0.1 s to 0.017 s. This shows that batching is essential for efficiency.

3.3 Service-Time Statistics

To support queueing-based performance modeling, we summarize empirical service times for both GPT-2 and DistilGPT-2 using their first and second moments. Figure 2 reports the mean service time with standard deviation for each model.

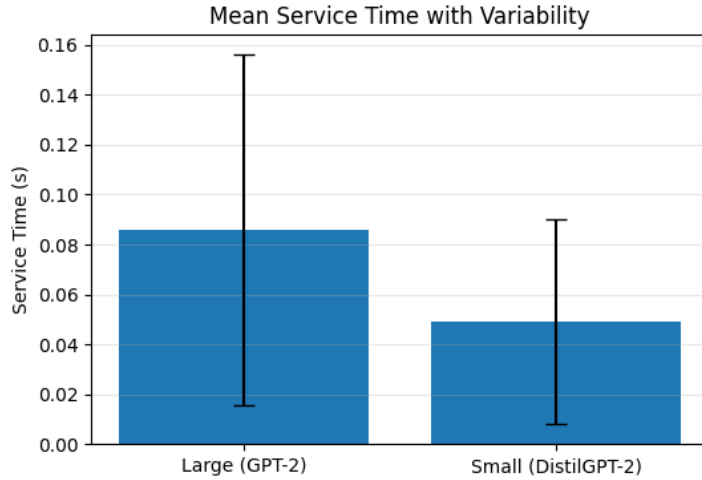


Figure 2: Moment-focused visualization: Mean $\pm$ Std.

GPT-2 exhibits higher mean latency and greater variability, while DistilGPT-2 is consistently faster and more stable. These statistics are sufficient to parameterize the M/G/1 queueing model used later in the analysis.

4 Online Optimization Algorithm and Design

The goal of GearShift LLM is not only to evaluate different routing policies, but to optimize system performance under explicit constraints. In particular, we aim to minimize average inference latency while maintaining acceptable output quality.

To achieve this, we formulate the problem as a policy optimization task over a discrete configuration space defined by:

- the request arrival rate λ ,
- the queue-length threshold T ,
- the switching strategy (deterministic or probabilistic).

For each configuration, we simulate an online inference service and measure system-level performance metrics, including average latency, fraction of requests handled by the small model, and expected output quality. This exhaustive grid search enables a systematic exploration of the latency–quality trade-off space.

Unlike static serving strategies, dynamic switching policies adapt model usage based on real-time load conditions. By explicitly evaluating these policies under increasing arrival rates, we identify configurations that offer the best latency reduction while respecting quality constraints. This optimization step translates empirical measurements and queueing behavior into actionable system design decisions.

4.1 Queue Simulator

We built a discrete-event simulator for a FIFO single-server queue, FIFO was chosen as it reflects fair request handling and aligns with M/G/1 assumptions, Jobs arrive according to a Poisson process with rate $\lambda \in \{0.5, 1.0, 2.0, 3.0\}$ requests per second. Each job is routed to a model based on the switching policy and receives a service time drawn from an exponential distribution parameterized by the empirical mean service time of the selected model.

We simulate 5000 requests per configuration.

4.2 Switching Policies

Two policy families were implemented:

- **Deterministic:** If queue length $\leq T$, use GPT-2; otherwise use DistilGPT-2.
- **Probabilistic:** Above threshold T , use DistilGPT-2 with probability 0.8.

Thresholds $T \in \{0, 2, 5, 10\}$ were tested, and simulation results show that lower thresholds switch too aggressively, causing unnecessary quality loss, while higher thresholds react too late and allow latency to increase significantly under higher arrival rates. Threshold $T = 2$ therefore represents the earliest point at which switching meaningfully reduces latency without noticeably degrading average quality, making it optimal across all tested arrival rates.

4.3 Model Usage vs Arrival Rate

Figure 3 shows how often the large model is used under $T = 2$.

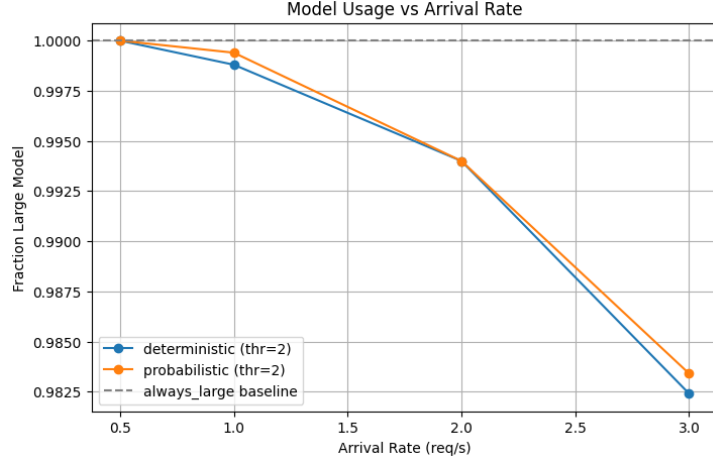


Figure 3: Fraction of large-model usage vs arrival rate.

At low arrival rates both policies behave like an “always-large” baseline. As traffic increases, both switch to the small model slightly more often, which helps keep latency controlled.

4.4 Latency Improvement from Dynamic Switching

To better isolate the impact of threshold selection, we measure latency improvement relative to a conservative reference configuration. Figure 4 reports the change in average latency achieved with threshold $T = 2$ as a function of arrival rate.

Results show that earlier switching yields increasing latency reductions as load grows, confirming that $T = 2$ represents an effective operating point where responsiveness improves without aggressive quality loss.

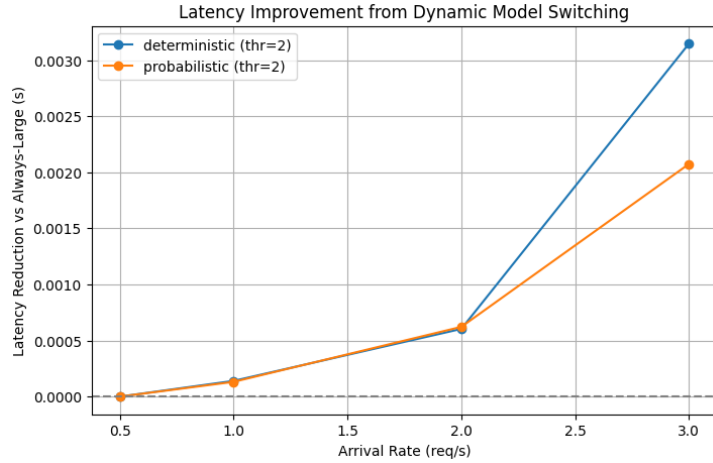


Figure 4: Latency reduction achieved by dynamic switching as a function of arrival rate

Latency increases smoothly with load, reflecting queueing effects. While absolute latency remains close to the large-model baseline, dynamic switching yields small but consistent reductions under higher arrival rates by preventing queue buildup.

4.5 Predictive M/G/1 Validation

We computed the predicted latency using the M/G/1 formula:

$$T = E[S] + \frac{\lambda E[S^2]}{2(1 - \lambda E[S])}.$$

Figure 5 compares predicted vs simulated latency.

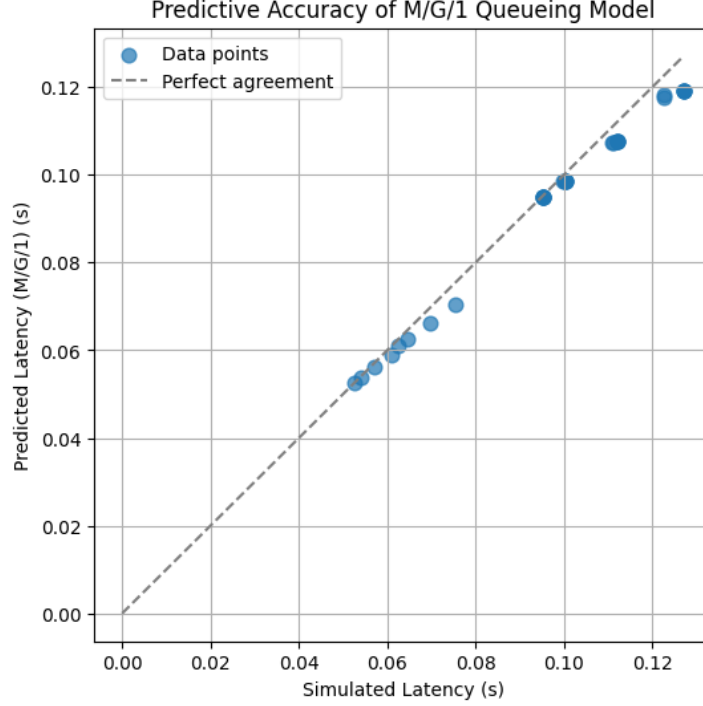


Figure 5: Predictive accuracy of the M/G/1 analytical model.

Points lie close to the diagonal, indicating that the M/G/1 model accurately captures average latency trends under dynamic model switching.

4.6 Optimal Policy Selection under Quality Constraints

Table 2 summarizes the best-performing switching policy for each arrival rate, selected by minimizing average latency while enforcing a minimum quality constraint.

Table 2: Best model-switching policy selected under a quality constraint ($\text{avg_quality} \geq 0.95$) for different arrival rates.

Arrival Rate (req/s)	Policy	Threshold	Avg. Latency (s)	Avg. Quality
0.5	Deterministic	2	0.089	1.000
1.0	Deterministic	2	0.093	0.999
2.0	Probabilistic	2	0.103	0.998
3.0	Deterministic	2	0.113	0.995

4.7 Modeling Assumptions

The proposed system model relies on several standard assumptions that are consistent with online inference services and queueing theory. First, incoming requests are assumed to arrive

according to a Poisson process, which models independent users generating requests at random times. Second, requests are served in a First-In-First-Out (FIFO) manner, ensuring fair treatment of users and enabling direct comparison with classical M/G/1 queueing theory.

Third, the system is modeled as a single-server queue, reflecting the use of a single GPU for inference. Finally, service times are allowed to follow a general distribution, as empirical measurements show significant variability depending on model choice, batch size, and prompt length. These assumptions collectively justify the use of an M/G/1 queueing model for performance prediction.

5 Conclusion

This project investigated GearShift LLM, a dynamic model-switching strategy for scalable generative AI inference under variable load. Through offline benchmarking, we demonstrated that batching plays a dominant role in reducing inference latency and that the large and small models exhibit clearly distinct service-time characteristics in both mean latency and variability.

Using empirical service-time statistics, we built a discrete-event simulator to study the behavior of an online inference service under different arrival rates and switching policies. The simulation results show that dynamic switching stabilizes latency growth under increasing load, while preserving high output quality by limiting the use of the smaller model when the system is lightly loaded.

A key finding of this study is the importance of threshold selection. Across all tested arrival rates, a moderate switching threshold ($T = 2$) consistently emerged as the most effective operating point. This threshold enables early intervention against queue buildup, yielding small but consistent latency reductions at higher load without aggressive quality degradation.

We further validated the system using an analytic M/G/1 queueing model parameterized by empirical service-time moments. Predicted latency closely matches simulation results, confirming that queueing theory provides an accurate and interpretable performance model for dynamic inference systems, even under model switching.

Overall, GearShift LLM demonstrates that modest architectural changes, grounded in empirical measurement and queueing analysis, can yield meaningful scalability benefits. By combining offline benchmarking, online simulation, constrained optimization, and analytic modeling, this project provides a principled framework for designing and evaluating scalable generative AI inference services.

Limitations: This study models a single-GPU inference server with FIFO scheduling and Poisson arrivals, and does not account for multi-GPU parallelism, priority-based scheduling, or non-stationary traffic patterns. Future work could extend the model to multi-server settings and explore adaptive thresholds under time-varying workloads.

6 References

1. Alec Radford et al., “Language Models are Unsupervised Multitask Learners”, OpenAI Technical Report, 2019.
2. Victor Sanh et al., “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter”, arXiv preprint arXiv:1910.01108, 2019.
3. Thomas Wolf et al., “HuggingFace’s Transformers: State-of-the-art Natural Language Processing”, Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2020.
4. Nils Reimers and Iryna Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”, arXiv preprint arXiv:1908.10084, 2019.
5. Leonard Kleinrock, *Queueing Systems, Volume I: Theory*, Wiley, 1975.
6. Mor Harchol-Balter, *Performance Modeling and Design of Computer Systems*, Cambridge University Press, 2013.
7. PyTorch Documentation, <https://docs.pytorch.org/docs/stable/index.html>
8. HuggingFace Transformers Documentation, <https://huggingface.co/docs/transformers>